

Bedtime Story Generator — Crash Course

Swarup Biswas

2026-07-09

Contents

Bedtime Story Generator — A Crash Course	1
Why this course exists, and how it's different	2
How each module is structured	2
Prerequisites	3
Where you work in this course	3
What we're building	4
Module 0 — Baseline reset	5
Module 1 — Swap Ollama for the Gemini API	8
Module 2 — Replace the textarea with a structured form	13
Module 3 — Compose the user prompt from form fields	17
Module 4 — Strengthen the system prompt for child safety + bedtime tone	21
Module 5 — Update the UI for parents reading aloud	25
Module 6 — Story library (re-hear yesterday's story)	28
Module 7 — Deploy: Vercel frontend + Render backend	33
Where to next	37
About this course	38

Bedtime Story Generator — A Crash Course

In this crash course, you take a working FastAPI Q&A app and convert it, one fundamental at a time, into a children's bedtime-story generator deployed to Vercel + Render. By the end, you have an app on the internet that lets a parent enter a child's name, characters, a setting, and a one-line plot, and returns a short, child-safe story written by Google's Gemini — with every story persisted so the child can re-hear yesterday's story without re-generating it.

The architecture in one line:

```
[ Browser ] → [ Vercel static frontend ] → [ Render FastAPI backend ] → [ Gemini  
↪ API ]  
  
                                     |  
                                     ↪ [ Render Postgres ]
```

This course is for adult learners who already shipped a FastAPI app — specifically, graduates of the Local LLM Question Log course in the same series. You probably haven't called a hosted LLM through an SDK with an API key, written a system prompt with safety constraints, deployed a split frontend/backend stack, or had to think carefully about cross-origin requests. You will, by Module 7.

The course is staged across 8 modules (0 through 7). Each module adds (and removes) exactly one fundamental capability over the previous one, and ends with a working, runnable app. You don't need to finish all 8 in one sitting — every module's checkpoint is a real app you can stop at, walk away from, come back to.

Why this course exists, and how it's different

There are a thousand “build a chatbot with the Gemini API” tutorials. This one is different in three ways:

- One idea per module. Module 1 is just “swap your local LLM for a hosted one via SDK.” Module 4 is just “production system prompts include safety constraints.” Not a bundle of features. Just the one thing. This gives you time to build a clean mental model of how each piece works before the next piece arrives.
- Conversion, not from-scratch. Module 0 is the V1 final state of the prior FastAPI Q&A course — copied byte-identical from `app-lego-blocks/dist/module_08_configuration/`. Every later module transforms that app: deletes some code, adds some code, ends with a still-working app. For every module past 0, expect to read about both halves — what’s removed (and why it was right for V1), what’s added (and why it’s right here. The transformation itself is the lesson; the V1 mental models you already have are what the course leverages.
- Minimal, readable code. You won’t see frameworks-on-frameworks, ceremony, or features the running app doesn’t need. No LangChain, no DI containers, no Pydantic-settings, no async, no streaming, no retry libraries. Each line earns its place by serving the module’s single fundamental.

Read on. By the end you’ll have built — and be able to defend, in your own words — a complete bedtime-story app on Vercel + Render.

How each module is structured

Every module from 1 onward follows the same shape. This is deliberate — adult learners build understanding faster when the surface is predictable and the variation is in the content, not the format.

Section	What it does for you
The notional machine	A runnable mental model of what the computer does when this code runs. Most tutorials skip this and leave you to reverse-engineer the runtime from syntax. We don’t.
The analogy	A concrete bridge from something you already know (a phone exchange, a passport check, a hire’s job description) to the abstract concept the module introduces. The analogy is named for where it breaks, too — that’s half the value.
What goes, what comes	Conversion-course halves: lines and files that disappear from the previous module, lines and files that take their place. Read the deletions first.
The code	The actual files. Full blocks, lifted verbatim from <code>slides/module_NN_*.md</code> — not diffs.
Trace it in execution order	A walkthrough of what happens first, second, third when the code runs — which is almost never the order the code is written in.
Predict before you run	A specific prediction prompt before you start the server. Surfacing your current mental model — and finding out where it’s wrong — is the single most powerful learning move in this whole course.
Run + verify	Concrete commands. The verify scripts (<code>scripts/verify_module_N.sh</code>) automate most of this.
Why this design	Why the code looks the way it does — what the alternatives were, what they would have cost, why the choice we made will be easier to maintain.
Defend It	One question to sit with before you move on. Don’t ask your AI partner to answer it. Reasoning through it yourself is the assessment.

Module 0 is slightly different because the app already exists (it's V1's final state) — the install steps and the anchor your baseline discipline replace the “the code” section.

Prerequisites

You need:

- Python 3.11 or newer (python --version should print 3.11.x or higher)
- Postgres (carries through V1; Module 6 introduces a `stories` table; Module 7 swaps to managed Postgres on Render)
- Ollama (Module 0 only — Module 1 replaces it with the Gemini API; you can stop Ollama from Module 1 onward)
- A Google AI Studio API key for Gemini (free tier; you'll create this in Module 1)
- A Vercel account and a Render account (Module 7; both have free tiers)
- About 5 GB of free disk space (Python venv + Ollama model + Postgres data)
- A code editor (VS Code, Antigravity, Cursor, JetBrains — any will do)
- Comfortable on the command line: `cd`, running scripts, reading error messages
- ~30–45 minutes per module for the first three modules; faster after that

You also need: the V1 Local LLM Question Log course's mental models. If you haven't shipped that course's V1 — server-as-receptionist, system prompt as a list of messages, env-var configuration with `KeyError` loud-fail — work through that course first. This crash course builds on those models rather than re-teaching them.

You do not need any prior experience with hosted LLM APIs, frontend frameworks (we don't use one), Docker, asyncio, retry libraries, or CI/CD pipelines beyond what Vercel and Render provide out of the box.

Where you work in this course

This course ships eight self-contained module folders under `dist/`. Each folder is the complete, runnable app for that module — no diff to apply, no files to write from scratch.

Module	Folder
Module 0 — Baseline reset	<code>dist/module_00_baseline/</code>
Module 1 — Swap Ollama for Gemini	<code>dist/module_01_swap_to_gemini/</code>
Module 2 — Replace textarea with form	<code>dist/module_02_form_input/</code>
Module 3 — Compose user prompt	<code>dist/module_03_prompt_composition/</code>
Module 4 — Strengthen system prompt	<code>dist/module_04_safety_system_prompt/</code>
Module 5 — Read-aloud UI	<code>dist/module_05_story_ui/</code>
Module 6 — Story library	<code>dist/module_06_story_library/</code>
Module 7 — Deploy	<code>dist/module_07_deploy_vercel/</code>

The shape of every module's Run + verify step is the same four lines:

```
cd dist/module_NN_<slug>           # move into the module's folder
source ../../venv/bin/activate      # activate the shared venv (one-time per terminal)
cp .env.example .env               # this module's env contract, with sensible defaults
uvicorn app.main:app --reload      # run THIS module's app
```

The shared venv/ lives at the repo root. You install Python dependencies once, at the start of Module 0, by running `pip install -r requirements.txt` from the repo root. The repo-root `requirements.txt` is a course-wide union of every module's dependencies — installing it once gives you a venv that runs all 8 modules without per-module reinstalls.

Per-module `dist/module_NN_*/requirements.txt` files still exist (Render reads from the Module 7 folder at deploy time, and Module 1's BEFORE/AFTER narration "we removed httpx because we no longer need it" shows up by reading those files). You don't `pip install` from them during the course — they're documentation, not a runtime step.

`scripts/verify_setup.sh` is the only thing you run from the repo root — it checks Postgres, Ollama, the database, and Python versions, all of which are global to your machine, not folder-specific. After it passes, every module's work happens inside that module's **dist/** folder.

Common mistake: running `uvicorn app.main:app --reload` from the repo root. The repo root's `app/` is the V1-final state of this course (Module 7's deploy-shape backend, no `/ route`, CORS-only). You'll see `404 Not Found on GET /`. The fix is `cd dist/module_NN_<slug>/` first.

What we're building

The final V1 app has four moving pieces:

The browser loads `index.html`, `style.css`, and `script.js` from Vercel's CDN. The page is a two-column layout: form on the left (four labelled fields — child name, characters, setting, plot — plus a "Past stories" panel that fills when the parent types a name), story on the right (rendered in serif at a generous size on a soft cream pane, capped at ~38 characters per line for read-aloud cadence).

The frontend on Vercel is three static files. There is no build step, no JS framework, no `package.json`. A single JavaScript constant `BACKEND_URL` points at the Render service URL.

The backend on Render is a long-running `uvicorn` process serving `app/main.py`. Three endpoints: `POST /story` (generate and persist), `GET /stories?child_name=...` (fetch recent saved stories), `GET /healthz` (Postgres ping). Sync handlers throughout — no `async def`. CORS middleware lets the cross-origin browser talk to the API. Every story call goes through `compose_story_prompt()` (turns form fields into the user prompt) and `call_gemini()` (sends the user prompt + a constant system prompt to the `gemini-2.5-flash-lite` model via the `google-genai` SDK), then through `save_story()` (one `INSERT` into the `stories` table). The backend responds with the generated story body.

The managed Postgres on Render holds one table — `stories` — with a composite index on `(child_name, created_at DESC)` so the per-child retrieval query runs against the index without scanning. Every column is named for what's there: `child_name`, `characters`, `setting`, `plot`, `body`, `model_name`, `created_at`. The `model_name` column means a future provider swap is a one-line change; the doctrine showed itself in V1 and pays off again here.

What we're not using and why:

- No LangChain — "your backend is a client of an SDK" is the lesson; an orchestration framework would hide it.
- No DI container — modules export functions; FastAPI is the only object.
- No Pydantic-settings — three plain `os.environ[...]` reads at module load are clearer.
- No retry/backoff library — surface API errors plainly first; backoff earns its place when a fundamental needs it.
- No streaming responses — full-response only, like V1.
- No `async def` — sync handlers throughout, carries from V1.
- No JS framework — plain HTML, plain CSS Grid, vanilla `fetch()`.
- No tests in V1 — testing is its own fundamental, deferred to a later curriculum.

[Browser] → [Vercel static frontend] → [Render FastAPI backend] → [Gemini
↪ API]

└→ [Render Postgres]

Module 0 — Baseline reset

The notional machine

Before any code runs, every dependency must be reachable. This course is a conversion course: Module 0 is the V1 final state of the prior FastAPI Q&A course, copied byte-identical. The “code” already exists; what you’re verifying is that the V1 stack you already shipped still runs the way you remember — Python 3.11, Postgres up and accepting connections, Ollama up and serving llama3.2, the interactions table present, the .env correctly pointing at all three. If any of those is wrong, every later module fails with errors that look like new bugs but are actually Module-0-debt.

The runtime model: when you run `uvicorn app.main:app --reload`, the Python interpreter reads `app/main.py`, which imports `app/services/ollama_service.py` and `app/services/interaction_service.py`, which read `os.environ["DATABASE_URL"]`, `os.environ["OLLAMA_BASE_URL"]`, and `os.environ["OLLAMA_MODEL"]` at import time. If any of those env vars is missing, the program crashes before uvicorn even binds the port. That loud-fail discipline is V1’s Module 8 lesson, and Module 0 is your last chance to confirm it before Module 1 changes which env vars matter.

The analogy

A pre-flight checklist. The pilot doesn’t take off and then check the flaps; they walk the plane on the ground. Module 0 is your walk: every dependency named, every probe green, before Module 1 starts taking things apart.

What goes, what comes

Nothing yet — Module 0 is the anchor. The whole point is that the code in `dist/module_00_baseline/` is byte-identical to `app-lego-blocks/dist/module_08_configuration/` — V1’s final state, frozen and shipped. If your editor diff between this folder and your edited V1 working tree shows any difference, you’re working from an edited V1, not the canonical baseline. The first thing the conversion course teaches is trust your starting point. If you’re not sure, re-pull the cohort repo to refresh `dist/module_00_baseline/`.

Install Python, Postgres, Ollama (if you don’t already have them)

Cohort students arriving from the FastAPI Local LLM Question Log course: you already have all three installed and verified from that course’s pre-flight — skip to “Clone or download the repo” below.

If you’re reading bedtime cold without the FastAPI course’s pre-flight, you need Python 3.11+ (3.12 is the sweet spot; 3.11–3.13 all work), Postgres 17, and Ollama with llama3.2 pulled — plus a conventional postgres superuser with password postgres and an empty database `llm_question_log`.

The canonical install walkthrough is the FastAPI course’s `docs/setup_walkthrough.md` — it covers macOS (Homebrew) and WSL2 Ubuntu (Windows learners) in one document, with a verification step after each tool. Work through that doc, then come back here.

Ollama is only needed for Module 0 (the V1 baseline still uses it). Module 1 swaps it out for the Gemini API; from Module 1 onward you can stop the Ollama service.

Clone or download the repo

```
git clone https://github.com/<user>/bedtime-story-generator.git
cd bedtime-story-generator
```

Create the shared venv and install all course dependencies (once per machine)

The shared venv/ lives at the repo root. Every dist folder activates it via `../../venv/bin/activate`.

```
python3 -m venv venv
source venv/bin/activate
```

(macOS / Linux / WSL2 Ubuntu — same commands. The (venv) prefix in your prompt is the visual confirmation.)

Now install all course dependencies in one shot. The repo-root requirements.txt is the union of every module's deps:

```
pip install -r requirements.txt
```

That's it for Python packages — you won't pip install again until you find a real new dependency to add. The shared venv now has fastapi, uvicorn, jinja2, httpx, psycpg, python-dotenv, and google-genai, which together cover Modules 0 through 7.

Prepare Module 0's .env

Module 0's app reads .env from dist/module_00_baseline/. Copy that folder's .env.example into a real .env:

```
cd dist/module_00_baseline
cp .env.example .env
```

Open dist/module_00_baseline/.env in your editor — the three V1 vars are already filled in with sensible local defaults:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2
```

If your local Postgres uses a different postgres password, edit DATABASE_URL to match. Otherwise, leave the file as-is.

(Module 1 will introduce a different .env.example — its contract drops the two Ollama vars and adds GEMINI_API_KEY. Each module's .env.example reflects what that module needs.)

Also export DATABASE_URL in your shell so the per-module psql "\$DATABASE_URL" ... verify commands work. The Python app reads .env via load_dotenv(), but that loads into the Python process's os.environ, not your terminal — so your shell still sees \$DATABASE_URL as empty unless you export it explicitly.

You have two options. Recommended: make it persistent so every new terminal auto-loads it (the course spans multiple modules across multiple sessions; you will close and reopen your terminal):

```
# macOS (zsh --- modern default):
echo 'export
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log' >>
~/.zshrc
source ~/.zshrc

# Linux / WSL2 Ubuntu (bash):
echo 'export
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log' >>
~/.bashrc
source ~/.bashrc
```

Fallback: one-time-per-terminal export if you'd rather not modify your shell rc file. You'll need to re-run this every time you open a fresh terminal:

```
export DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log
```

(macOS zsh / Linux bash / WSL2 Ubuntu bash — same line.)

Whichever option you choose covers every module's psql verify (Module 0's interactions queries, Module 6's stories queries, the deploy-time migration in Module 7, etc.).

Symptom if the var goes missing: `psql: ... FATAL: database "<your-username>" does not exist.` That's `psql` falling back to the OS username because `$DATABASE_URL` was empty. Most common cause: you opened a fresh terminal (look for the "The default interactive shell is now `zsh`" macOS banner at the top of your prompt) and the one-time export from a previous session is gone. Fix: re-export, or set it persistently per the recommended option above.

Confirm — or bootstrap — the database

The `interactions` table is V1's table. It carries unchanged through Modules 0 and 1, sits idle through Modules 2–5, and is dropped in Module 6's migration when the new `stories` table takes its place. If you completed the prior V1 course on this machine, the database and the table already exist — you do not need to recreate them.

`verify_setup.sh` will tell you which case you're in. If you saw both:

- ✓ Database `llm_question_log` reachable as user 'postgres'
- ✓ Table `interactions` exists

skip ahead to "Trace what `verify_setup.sh` actually does". Your V1 database carries through.

Only if **`verify_setup.sh`** reports the database or table missing (fresh Postgres install, or a different machine from the V1 course), bootstrap from the SQL files at the repo root:

```
# database missing --- create it
psql -h localhost -U postgres -c "CREATE DATABASE llm_question_log;"

# table missing --- apply the V1 schema
psql "$DATABASE_URL" -f sql/001_create_interactions.sql
```

(`sql/000_create_database.sql` is a fallback for installs that don't put `createdb` on `PATH`; the `psql ... CREATE DATABASE` above does the same thing.)

Trace what **`verify_setup.sh`** actually does

The Module 0 verify script runs five probes, each `|| fail` so the first red `✗` stops the script:

1. `python --version` matches 3.11 or higher.
2. `pg_isready -h localhost` returns success — Postgres is running, not just installed.
3. The `interactions` table exists in the database `DATABASE_URL` points at.
4. `ollama list` succeeds and lists `llama3.2`.
5. A smoke `curl` to `http://localhost:11434/api/chat` returns a chat completion — proves Ollama is responding, not just listed.

Each probe answers a different "installed but not running" failure mode the course has hit in past cohorts.

Predict before you run

Before running the verify script, write down (mentally or literally):

- Which of the five probes do you predict will go red on your machine? Be honest — if you skipped the `brew services start postgresql@17` step on Mac, probe 2 will fail.
- For each likely red, what's your guess at the fix? Probe 2: "start Postgres." Probe 4: "`ollama pull llama3.2`."
- What would happen if you skipped Module 0 verification entirely and jumped straight into Module 1? (Hint: the failures would arrive in Module 1's code in misleading ways — a `psycpg.OperationalError` from a Postgres that's installed but stopped looks like a code bug, not a setup bug.)

Run + verify

From the repo root (not a dist folder — `verify_setup.sh` checks global services that aren't folder-specific):

```
./scripts/verify_setup.sh
```

(macOS / Linux / WSL2 Ubuntu — same command.)

Expected: five green ✓ checks, ending with “All checks passed. You’re ready for Module 1.”

Then run the V1 app one more time before we start dismantling it. Module 0’s app lives in `dist/module_00_baseline/` — you should still be in that folder from the install step above. If you opened a fresh terminal, re-`cd` and re-activate the shared venv first:

```
cd dist/module_00_baseline          # only if you're not already here
source ../../venv/bin/activate      # only if `(venv)` isn't in your prompt

uvicorn app.main:app --reload
```

If uvicorn crashes with **KeyError: 'OLLAMA_BASE_URL'** at import time (before binding the port), the `.env` file is missing from this folder. The fix is `cp .env.example .env` (covered above), then re-run uvicorn. That’s V1’s loud-fail discipline working correctly — Module 1 will rediscover the same shape with **KeyError: 'GEMINI_API_KEY'**.

Open `http://localhost:8000` in a browser. Type a question into the textarea (“What is FastAPI?”) and submit. You see the answer rendered, plus the recent question/answer history below it. That’s the V1 Q&A app — same shape you shipped at the end of the prior course. Module 1 deletes the Ollama wiring; Module 2 deletes the textarea; Module 6 drops the interactions table. By Module 7 nothing of this V1 surface remains. This is your last look at V1.

Why this design

Module 0’s job is to anchor your baseline. Three reasons it’s worth a whole module:

1. Conversion courses depend on a known starting point. If two students start from two different “edited V1s,” the same Module 1 instructions produce two different outcomes and the cohort drifts. `dist/module_00_baseline/` (byte-identical to `app-lego-blocks/dist/module_08_configuration/`) is the single canonical source — re-pull the cohort repo if your copy has drifted.
2. “Installed but not running” is the most common dry-run failure mode. A `python --version` check passes when Python is installed; it doesn’t check that Postgres or Ollama have started. The five-probe triad explicitly verifies running services, not just installed binaries.
3. The loud-fail rediscovery. When you delete `OLLAMA_BASE_URL` from `.env` in Module 1 and replace it with `GEMINI_API_KEY`, the failure mode of “missing env var → app refuses to start” is the same failure mode V1’s Module 8 taught. Module 0 is your last chance to feel that discipline before it shows up in a slightly different shape.

Defend It

Why is anchoring Module 0 to `app-lego-blocks/dist/module_08_configuration/` (and not your edited V1 working tree) worth its own module — instead of just being a one-line “make sure your V1 still runs” check at the start of Module 1?

Don’t ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 1 — Swap Ollama for the Gemini API

The notional machine

A hosted LLM is a server you don’t run, addressed by an API key, accessed through a vendor SDK. Where V1’s Ollama call was an HTTP POST to `http://localhost:11434/api/chat` with a hand-built JSON body, Module 1’s Gemini call is `client.models.generate_content(model=..., contents=..., config=...)`

— a Python method call. The SDK opens the TLS connection, authenticates with your GEMINI_API_KEY, sends the request to Google's servers, parses the response, and hands you back a typed object whose .text attribute is the model's reply.

You no longer run the LLM process. The model itself runs on Google's infrastructure; your laptop is purely a client. That changes three things you need to hold in your head:

1. Latency. A typical call takes 0.5–3 seconds (TLS handshake + network + model time). Localhost was tens of milliseconds.
2. Quota. The free tier caps you at a low requests-per-minute rate on gemini-2.5-flash-lite (check <https://ai.google.dev/pricing> for the current limit). Hit the cap and you get HTTP 429s — a real failure mode, not a bug.
3. Authentication. A single env var, GEMINI_API_KEY, is the entire access surface. If it's missing, the program refuses to start (loud-fail). If it's wrong, every call returns a 4xx.

The analogy

V1's Ollama was cooking on your own stove — you owned the model, the process, the port. Module 1's Gemini is ordering from a restaurant kitchen — someone else owns the stove; you have a phone number (the API key) and a menu (the SDK methods). You no longer worry about gas or pots; you do worry about whether the kitchen is open and how much they charge per dish (quota).

Where the analogy breaks: a restaurant won't refuse to serve you because of a missing reservation; the Gemini API absolutely will refuse to start your app because a single string isn't in os.environ. The loud-fail discipline is part of your code, not Google's.

What goes, what comes

Goes: - The whole file app/services/ollama_service.py — every line was load-bearing for V1's localhost Ollama call, and none of those lines map onto an SDK call. - httpx from requirements.txt — its only two callers (the Ollama client and the Ollama branch of /healthz) are both deleted. - OLLAMA_BASE_URL and OLLAMA_MODEL from .env.example — the SDK owns the endpoint; the model name pairs with the system prompt and stops being environment. - The Ollama branch of /healthz — pinging a paid hosted API on every health-check costs money and burns quota. Health-checks for paid SDKs belong outside the request path.

Comes: - A new file app/services/gemini_service.py with call_gemini(question) — the SDK call, narrowly catching genai_errors.APIError and translating to HTTPException(502). - google-genai>=1.0.0 in requirements.txt. (Note: not google-generativeai; that older package was deprecated by Google in late 2025.) - GEMINI_API_KEY=... in .env.example, with the comment that you generate one at <https://aistudio.google.com/apikey>. - A one-line import swap in app/services/interaction_service.py from OLLAMA_MODEL to GEMINI_MODEL — the V1 design intent (model_name as a string column on interactions) pays off as a one-line change rather than a schema migration.

User-visible behaviour: identical. You ask, you get an answer, you see it in history. The wiring underneath is now hosted-API.

The code

app/services/gemini_service.py (new file):

```
import os
from google import genai
from google.genai import types
from google.genai import errors as genai_errors
from fastapi import HTTPException

GEMINI_API_KEY = os.environ["GEMINI_API_KEY"]
GEMINI_MODEL = "gemini-2.5-flash-lite"
SYSTEM_PROMPT = (
```

```

    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)

client = genai.Client(api_key=GEMINI_API_KEY)
generate_config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)

def call_gemini(question: str) -> str:
    try:
        response = client.models.generate_content(
            model=GEMINI_MODEL,
            contents=question,
            config=generate_config,
        )
        return response.text
    except genai_errors.APIError:
        raise HTTPException(status_code=502, detail="Gemini is not reachable.")

```

app/main.py (three edits):

```

@app.post("/ask", response_model=AskResponse)
def ask(payload: AskRequest):
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="Please enter a question.")
    answer = call_gemini(question) # ← was call_ollama
    save_interaction(question, answer)
    return AskResponse(answer=answer, history=fetch_recent_history())

@app.get("/healthz")
def healthz():
    status = {"postgres": False} # ← was {"ollama": False,
    ↪ "postgres": False}
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
                status["postgres"] = True
    except psycopg.Error:
        pass
    return status

```

app/services/interaction_service.py (one import line + one symbol):

```

from app.services.gemini_service import GEMINI_MODEL # ← was OLLAMA_MODEL from
↪ ollama_service

# inside save_interaction:
cur.execute(
    "INSERT INTO interactions (question, answer, model_name) "
    "VALUES (%s, %s, %s)",
    (question, answer, GEMINI_MODEL), # ← was OLLAMA_MODEL
)

```

requirements.txt:

```
- httpx==0.27.*
+ google-genai>=1.0.0
```

.env.example:

```
- OLLAMA_BASE_URL=http://localhost:11434
- OLLAMA_MODEL=llama3.2
+ GEMINI_API_KEY=your-google-ai-studio-key-here
```

Trace it in execution order

When the browser POSTs /ask with {"question": "What is FastAPI?"}:

1. At program start (already happened): Python imports app/main.py, which imports app/services/gemini_service.py. That module reads os.environ["GEMINI_API_KEY"] immediately — if the key is missing, KeyError raises and uvicorn never finishes starting. If the key is present, the file constructs client = genai.Client(api_key=GEMINI_API_KEY) once, and generate_config once, both at module level.
2. Request arrives: uvicorn accepts the TCP connection, hands the HTTP request to FastAPI.
3. Routing + validation: FastAPI matches /ask to the ask() function and runs Pydantic validation on the body against AskRequest. If the body is missing the question field, the request 422s here without ask() running.
4. Handler logic: ask() strips the question, raises 400 if blank, then calls call_gemini(question).
5. SDK call: call_gemini invokes client.models.generate_content(...). The SDK opens (or reuses) a TLS connection to Google, authenticates with the API key, sends the request, blocks until Google replies, parses the response, returns a typed object.
6. Error catch: if the SDK raised genai_errors.APIError (network down, 4xx, 5xx, quota exhausted), the catch block converts it to HTTPException(502) and the function raises.
7. Persistence: save_interaction(question, answer) opens a Postgres connection, INSERTs the row with model_name=GEMINI_MODEL, commits, closes.
8. Response: fetch_recent_history() SELECTs the recent ten rows, builds an AskResponse object, FastAPI serialises to JSON, the response goes back to the browser.

The order matters: client = genai.Client(...) runs once at module import. Putting it inside call_gemini would re-create the client on every request — slow, wasteful, and a sign you don't understand the SDK's lifecycle.

Predict before you run

Before running uvicorn app.main:app --reload, write down (mentally or literally):

- What happens if you start uvicorn with GEMINI_API_KEY missing from your environment? Will it fail at import time, at first request time, or never?
- What happens if you start it with the key set to an empty string?
- What happens if Postgres is stopped but Gemini is reachable? (Hint: think about which line in the request flow runs first.)
- What's the response body of GET /healthz going to look like compared to V1's? Trace the dictionary.
- The first /ask call after uvicorn starts: will it be faster or slower than the second? Why?

Run, then compare your predictions to reality.

Run + verify

```
cd dist/module_01_swap_to_gemini # from the repo root, or `cd
↪ ../module_01_swap_to_gemini` from Module 0
source ../../venv/bin/activate # only if `(venv)` isn't in your prompt

cp .env.example .env # this folder's .env contract is GEMINI_API_KEY-only
↪ --- Ollama vars are gone
```

```
# edit .env to paste your real key from https://aistudio.google.com/apikey
uvicorn app.main:app --reload
```

Then verify:

```
# /ask now hits Gemini:
curl -s -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"question":"What is FastAPI in one sentence?"}'
# Expected: {"answer":"<coherent sentence>","history":[...]}

# /healthz no longer reports Ollama:
curl -s http://localhost:8000/healthz
# Expected: {"postgres":true}      (no "ollama" key --- that branch is gone)

# Postgres recorded the new model name.
# Reminder: `DATABASE_URL` must be exported in your *shell*. load_dotenv() loads it
# into the Python process's os.environ, not your terminal. If you opened a fresh
# terminal since Module 0 (the macOS "default interactive shell is now zsh" banner
# is the giveaway), the export from that earlier session is gone --- re-export now,
# or set it persistently in your shell rc file (see Module 0 setup, recommended
# option). One-line re-export:
#   export
↪ DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log
# Symptom if missing: `psql: ... FATAL: database "<your-username>" does not exist`.
psql "$DATABASE_URL" -tAc \
  "SELECT model_name FROM interactions ORDER BY id DESC LIMIT 1"
# Expected: gemini-2.5-flash-lite

# All-in-one:
./scripts/verify_module_1.sh
```

The first call may take a couple of seconds (TLS handshake + first-token latency); subsequent calls are faster. The answer's content is non-deterministic — the verify script checks shape and the `model_name` column, not the wording.

Why this design

Three doctrine choices in this module are worth narrating:

GEMINI_MODEL is a hardcoded constant, not an env var. V1 made `OLLAMA_MODEL` env-driven because Ollama legitimately varies per machine. Here, the model name pairs with the system prompt — they're one design decision (changing the model changes prompt behaviour), so versioning them together is honest. `GEMINI_API_KEY` stays env-driven because keys legitimately differ per environment (your free-tier key, the cohort instructor's key, the Render production key). PRD §1.5 rule 2: take away, not add. A `Settings/BaseSettings` class for two settings is too few fields to earn the abstraction.

`/healthz` drops the Ollama branch and does not replace it with a Gemini probe. Pinging a paid hosted API on every health-check costs money and burns quota. Production health-checks for paid SDKs belong outside the request path (a startup smoke-test, not on every probe). The Postgres branch stays because Postgres is local and free to ping. PRD §1.5 rule 2: take away, not add.

The `try/except genai_errors.APIError` block earns its place even though the doctrine forbids "try/except that catches and re-raises with no added value." The added value is the translation: SDK-specific errors become HTTP 502 for the browser. That's value the catch is doing — converting an internal exception type into the boundary's error contract. The catch is also narrow (`genai_errors.APIError`, not bare `Exception`), so unrelated

code bugs still surface as 500s. PRD §1.5 prohibitions list: catch the narrowest exception that expresses the failure mode.

By the end of this module, you've replaced the V1 LLM client with a hosted SDK, dropped a runtime dependency (Ollama), and learned to read the SDK's lifecycle. Module 2 builds on this by changing what the user sends — the textarea becomes a structured form, and /ask becomes /story.

Defend It

What does calling Gemini through the *google-gemini* SDK give us that an *httpx.post()* to *generativelanguage.googleapis.com* — the shape we used for Ollama — wouldn't?

Don't ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 2 — Replace the textarea with a structured form

The notional machine

Pydantic validation is a wall the framework builds between the network and your handler. When a request body comes in, FastAPI parses it as JSON, then matches each field against the declared model (StoryRequest). If a field is missing or the wrong type, the framework returns 422 without your handler ever running. Your handler receives a fully-typed object with attributes — `payload.child_name`, `payload.plot` — and never has to do `dict.get("field", "")` boilerplate.

In this module, that wall changes shape. V1 had a one-field `AskRequest(question: str)`; Module 2 has a four-field `StoryRequest(child_name, characters, setting, plot)`. The wall now catches different things — “you didn't send *setting*” instead of “you didn't send *question*”. The validation layer, the handler's business-rule check, and the browser's required attribute now form three layers each catching what the others miss.

The analogy

A passport check at airport immigration. Three checks at three counters: the boarding-card scan (browser required — you can't even leave the gate without it), the document scan (Pydantic 422 — wrong shape, sent home before the immigration officer sees you), the officer's manual look (handler's `if not ... .strip()` — passport is the right shape but the photo's been scribbled on). Each catches what the others miss. None of the three is redundant.

Where the analogy breaks: an airport's three checks are sequential (you can't reach the officer without passing the document scan); a web app's three checks are parallel-trigger — a malicious client can skip the browser layer entirely and send raw JSON, but Pydantic and the handler still run.

What goes, what comes

Goes: - Three Pydantic classes — `AskRequest`, `AskResponse`, `Interaction` — gone from `app/schemas.py`. - The whole file `app/services/interaction_service.py` — Q&A pairs are gone, so `save_interaction` and `fetch_recent_history` have no callers and become dead code. - The `/ask` and `/history` endpoints in `app/main.py` — replaced by `POST /story`. - The textarea, the `loadHistory()` and `renderHistory()` JS, and the history `<ol>` from `app/templates/index.html`. - The page title and `<h1>` flip from “Local LLM Question Log” to “Bedtime Story Generator”.

Stays: - `app/services/gemini_service.py` — unchanged. The user prompt is still passed through `call_gemini(prompt)`; the system prompt is still V1's Q&A leftover (Module 4 rewrites it). - The `interactions` Postgres table is left in the database, uncalled. Module 6 drops it. Not dropping in Module 2 keeps the rollback story honest (running Module 1's `dist/module_01_swap_to_gemini/` should still find a working DB) and keeps “schema deletion” as Module 6's lesson. - `escapeHtml()` in the template — still needed for rendering the story body safely.

Comes: - Two Pydantic classes — `StoryRequest(child_name, characters, setting, plot)` and `StoryResponse(story)` — in `app/schemas.py`. - A `POST /story` handler in `app/main.py` with an inline (deliberately ugly) f-string prompt and a comment promising Module 3 will lift it. - A four-field HTML form in `index.html`, a `#story-output` div, and a `renderStory()` function that splits on blank lines and wraps paragraphs in `<p>` tags.

The code

`app/schemas.py`:

```
from pydantic import BaseModel

class StoryRequest(BaseModel):
    child_name: str
    characters: str
    setting: str
    plot: str

class StoryResponse(BaseModel):
    story: str
```

Each field carries its meaning in its name. A future reader (you, two weeks later) sees `child_name` and knows what's there without context.

`app/main.py`:

```
app = FastAPI(title="Bedtime Story Generator")  # ← was "Local LLM Question Log"

from app.schemas import StoryRequest, StoryResponse
from app.services.gemini_service import call_gemini
# (interaction_service is no longer imported --- the file no longer exists)

@app.post("/story", response_model=StoryResponse)
def story(payload: StoryRequest):
    if not payload.child_name.strip() or not payload.plot.strip():
        raise HTTPException(
            status_code=400,
            detail="Please fill in at least the child's name and the plot.",
        )
    # Module 3 lifts this inline f-string into compose_story_prompt(payload).
    prompt = (
        f"Write a short bedtime story for a child named {payload.child_name}. "
        f"Characters: {payload.characters}. "
        f"Setting: {payload.setting}. "
        f"Plot: {payload.plot}."
    )
    return StoryResponse(story=call_gemini(prompt))
```

`/healthz` is unchanged from Module 1 (postgres-only). `/ask` and `/history` are removed — FastAPI returns 404 for those URLs, which is honest.

The inline f-string is deliberately ugly. The single comment — `# Module 3 lifts this inline f-string into compose_story_prompt(payload).` — earns its place because it tells a future reader (or a future AI partner) why the ugliness is intentional. Without that comment, an AI partner would (correctly, by general best practice) suggest the refactor; the comment is the explicit don't do that yet.

app/templates/index.html (substantive shape):

```
<title>Bedtime Story Generator</title>
<h1>Bedtime Story Generator</h1>

<form id="story-form">
  <label>Child's name <input name="child_name" required></label>
  <label>Characters <input name="characters" required></label>
  <label>Setting <input name="setting" required></label>
  <label>Plot <input name="plot" required></label>
  <button type="submit">Generate story</button>
</form>

<div id="story-output"></div>

<script>
document.getElementById("story-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const form = e.target;
  const body = {
    child_name: form.child_name.value,
    characters: form.characters.value,
    setting: form.setting.value,
    plot: form.plot.value,
  };
  const r = await fetch("/story", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(body),
  });
  const data = await r.json();
  renderStory(data.story);
});

function renderStory(text) {
  const out = document.getElementById("story-output");
  out.innerHTML = text.split(/\n\n+/).map(p => `<p>${escapeHtml(p)}</p>`).join("");
}
</script>
```

The form's four fields match the four Pydantic fields by name. The JSON body is built by reading `form.child_name.value` etc., so a reader can trace name-equality across HTML → JS → Pydantic in one glance.

Trace it in execution order

When a parent submits the form:

1. Browser pre-flight: HTML's required attribute fires first. If any field is empty, the browser refuses to POST and shows its native validation message. The request never leaves the browser.
2. POST arrives at FastAPI: assuming the browser did POST, the handler matches `POST /story` and runs Pydantic validation on the body against `StoryRequest`.
3. Pydantic check: every field is required-by-default (no `Optional[str]`, no defaults). Missing or wrong-typed → 422 with a list of every offence. The handler does not run.
4. Handler runs: Pydantic delivers payload typed; the handler's first check is `if not payload.child_name.strip()` or `not payload.plot.strip()`. This catches a malicious or buggy client that sent JSON with whitespace in required fields. → 400 with a friendly message.

5. Prompt construction: the inline f-string concatenates the four fields into one string. Note: no `.strip()` here yet — Module 3 adds that.
6. Gemini call: `call_gemini(prompt)` runs (the same SDK call as Module 1), returns the story body.
7. Response: `StoryResponse(story=...)` is built; FastAPI serialises to JSON; the browser receives it.
8. Browser render: `renderStory(data.story)` splits the text on blank lines and wraps each paragraph in a `<p>` tag, escaping HTML to be safe against any LLM-injected markup.

Three validation layers run in order: browser → Pydantic → handler. Each catches what the others miss; this is belt-and-braces, not redundancy.

Predict before you run

- POST `/story` with `{}` (empty body). Will it 422 (Pydantic) or 400 (handler)? Why?
- POST `/story` with a complete-but-whitespace body — `{"child_name": " ", "characters": "x", "setting": "y", "plot": "z"}`. Will Pydantic accept it? Will the handler? What's the status code and message?
- POST `/ask` (the V1 endpoint). What status code do you predict?
- Open the browser at `/`. Fill three of four fields; click submit. Does the request fire?

Run + verify

```
cd ../module_02_form_input          # from Module 1's folder, or `cd
↳ dist/module_02_form_input` from the repo root
source ../../venv/bin/activate      # if not already active in this terminal
# No new dependencies; no schema migration to run.
cp .env.example .env               # if you don't already have one in this folder; same
↳ GEMINI_API_KEY contract as Module 1
uvicorn app.main:app --reload
```

Open `http://localhost:8000`. Fill the form (child_name=Aisha, characters=an owl and a fox, setting=enchanted forest, plot=looking for the moon) and click Generate story.

```
# /story does the work:
curl -s -X POST http://localhost:8000/story \
  -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"an owl and a fox",
      "setting":"enchanted forest","plot":"looking for the moon"}'
# Expected: {"story":"<a coherent short bedtime story>"}

# Pydantic 422 on missing required:
curl -s -X POST http://localhost:8000/story -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha"}'
# Expected: 422 listing every missing field

# Handler 400 on blank required:
curl -s -X POST http://localhost:8000/story -H "Content-Type: application/json" \
  -d '{"child_name":"","characters":"x","setting":"y","plot":"z"}'
# Expected: 400 with "Please fill in at least the child's name and the plot."

# Old endpoints are 404:
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/ask -d '{}'
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/history
# Expected: 404, 404

# All-in-one:
./scripts/verify_module_2.sh
```


Why this design

The inline f-string is deliberately ugly because Module 3 owns the lift. Refactoring it into `compose_story_prompt(payload)` here would make the handler cleaner and would bundle Module 3's single fundamental ("prompt composition is itself a piece of code with rules and care") into Module 2 — leaving Module 3 with nothing to teach. PRD §1.5 rule 1 (simplicity over demonstration) and rule 3 (YAGNI / Rule of Three): a helper called from one place doesn't earn the abstraction yet.

interaction_service.py is deleted, not commented out, not parked. Code with no caller is dead code; doctrine says delete. Module 6 will create `app/services/story_service.py` with `save_story(...)` and `fetch_recent_stories(...)` — different functions, different table, different domain. Keeping the V1 file would lie about the codebase's shape. PRD §1.5 rule 2 (take away, not add).

Three validation layers are not redundancy. Browser required exists to give immediate UX feedback. Pydantic 422 exists because a malicious or buggy client can skip the browser. The handler's `.strip()` check exists because Pydantic counts whitespace as a valid string. Each layer catches a class of input the others can't. This belt-and-braces is what production-shape input validation actually looks like.

The `interactions` table sits in Postgres unused through Modules 2–5. Some students want to drop it now. Don't — Module 6's migration owns the schema deletion, and "schema deletion" is its own teachable lesson, not a Module-2 cleanup.

Defend It

Why is the inline `prompt = f"Write a short bedtime story..."` block deliberately ugly in this module instead of being lifted into a `compose_story_prompt()` helper from the start?

Don't ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 3 — Compose the user prompt from form fields

The notional machine

A prompt is not a string. It's a designed object that happens to be expressed as a string. Design shows up in three places: which fields go in, what whitespace separates them, and what labels frame them. Module 2 had the user prompt as an inline f-string in the HTTP handler — visible only by reading the handler. Module 3 lifts it into a function in its own file. The function is callable from a Python REPL with a hand-built `StoryRequest`, so you can iterate the prompt without `uvicorn` or a Gemini key.

The runtime model: a pure function `compose_story_prompt(req: StoryRequest) -> str`. No I/O, no side effects, deterministic. Same input always produces the same output. It's the most testable shape a function can take, even though V1's doctrine has no tests.

The analogy

The handler used to be a person trying to do five things at once — validate, build the prompt, call the model, save the result, return — and each was tangled with the others. The lift is taking the prompt-building out of the handler's hands and putting it on a designated kitchen counter. The handler still calls it. But the prompt-building has its own surface, its own file, its own set of design choices, and you can iterate it without the rest of the kitchen running.

What goes, what comes

Goes: - The inline f-string in `app/main.py`'s `/story` handler. - The promissory comment `# Module 3 lifts this inline f-string into compose_story_prompt(payload)`. - The single-line space-glued shape — replaced by multi-line labelled sections with `.strip()` on every value.

Stays: app/schemas.py, app/services/gemini_service.py, app/templates/index.html, requirements.txt, .env.example. This is a pure refactor; no schema, no SDK contract, no UI, no env var changes.

Comes: - A new file app/prompt.py (peer to app/schemas.py, not under services/). - One import in app/main.py: from app.prompt import compose_story_prompt. - A handler that shrinks from ~12 lines to ~6 and reads as a sentence: validate → compose → call → return.

The code

app/prompt.py (new file):

```
from app.schemas import StoryRequest

def compose_story_prompt(req: StoryRequest) -> str:
    return (
        f"Write a short bedtime story for a child named {req.child_name.strip()}. \n"
        f"\n"
        f"Characters: {req.characters.strip()} \n"
        f"Setting: {req.setting.strip()} \n"
        f"Plot: {req.plot.strip()} \n"
    )
```

app/main.py — handler before (Module 2):

```
@app.post("/story", response_model=StoryResponse)
def story(payload: StoryRequest):
    if not payload.child_name.strip() or not payload.plot.strip():
        raise HTTPException(
            status_code=400,
            detail="Please fill in at least the child's name and the plot.",
        )
    # Module 3 lifts this inline f-string into compose_story_prompt(payload).
    prompt = (
        f"Write a short bedtime story for a child named {payload.child_name}. "
        f"Characters: {payload.characters}. "
        f"Setting: {payload.setting}. "
        f"Plot: {payload.plot}."
    )
    return StoryResponse(story=call_gemini(prompt))
```

After (Module 3):

```
from app.prompt import compose_story_prompt

@app.post("/story", response_model=StoryResponse)
def story(payload: StoryRequest):
    if not payload.child_name.strip() or not payload.plot.strip():
        raise HTTPException(
            status_code=400,
            detail="Please fill in at least the child's name and the plot.",
        )
    return StoryResponse(story=call_gemini(compose_story_prompt(payload)))
```

The handler is now four operations, each with a name. You can read it once and say what it does; you couldn't before, because the inline f-string forced you to mentally reconstruct the prompt before you could see what the handler was for.

Trace it in execution order

When a /story request arrives:

1. Pydantic validates the body against StoryRequest. (Same as Module 2.)
2. The handler's `if not ... .strip()` check runs against `payload.child_name` and `payload.plot`. If blank → 400. (Same as Module 2.)
3. New: `compose_story_prompt(payload)` is called. It reads `payload.child_name`, `.characters`, `.setting`, `.plot` — strips each — interpolates into the multi-line template — returns the resulting string.
4. `call_gemini(prompt)` is called with the composed prompt. The SDK call is the same as before; what's different is who built the string.
5. The response is built and returned. (Same as Module 2.)

Note the order: the handler's `.strip()` check is on the raw payload field (catches whitespace-only inputs at the boundary). The compose function's `.strip()` is on the values that go into the prompt (normalises leading/trailing whitespace before the model sees them). Two strips for two reasons.

Predict before you run

- Run the same form payload twice through Module 2's app and Module 3's app. Will the stories be identical? Why or why not?
- In a Python REPL, build a `StoryRequest(child_name=' Aisha ', characters='owl', setting='forest', plot='moon')` and call `compose_story_prompt(req)`. What does the output look like? Where do the spaces around Aisha go?
- If a future module (say Module 5) wanted to A/B-test two different prompt templates, where would the change happen — in the handler, in the compose function, in `gemini_service`, or somewhere else?

Run + verify

```
cd ../module_03_prompt_composition
source ../../venv/bin/activate
cp .env.example .env # if you don't already have one in this folder
uvicorn app.main:app --reload

# Behaviour-equivalence: /story still returns a story.
curl -s -X POST http://localhost:8000/story \
  -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"an owl and a fox",
      "setting":"enchanted forest","plot":"looking for the moon"}'
# Expected: {"story":"<a coherent short bedtime story>"}

# The function is now testable in isolation --- no uvicorn, no Gemini key needed:
python -c "
from app.schemas import StoryRequest
from app.prompt import compose_story_prompt
req = StoryRequest(child_name=' Aisha ', characters='an owl', setting='forest',
    ↪ plot='moon')
print(compose_story_prompt(req))
"
# Expected: a multi-line string starting with 'Write a short bedtime story for a child
    ↪ named Aisha.'
# Note: the leading/trailing spaces around 'Aisha' are gone --- the .strip() did its
    ↪ job.

# All-in-one:
./scripts/verify_module_3.sh
```

Why this design

The new file is **app/prompt.py**, peer to **app/schemas.py**, not **app/services/prompt.py**. The **services/** folder is reserved for code that talks to external concerns — **gemini_service.py** talks to the Gemini SDK; previously **interaction_service.py** talked to Postgres. **compose_story_prompt** is pure string manipulation; classifying it as a service would mis-name what it is and pull future modules to put more pure-logic things into **services/** for no reason. PRD §1.5 rule 4 (build for maintainability — short, clear names that describe what's there).

The lifted function isn't just the inline f-string in a **def**. It also adds (a) `.strip()` on every field value and (b) multi-line structure with newlines between labelled sections. Both changes demonstrate that “rules and care” means more than “lifted into a function” — they show that a prompt is a designed object with whitespace, structure, and normalisation choices. The model often produces visibly more-paragraphed output for the multi-line prompt than for Module 2's single-line one, even with identical inputs. That's the lesson — prompt structure influences output structure.

The function signature is **(req: StoryRequest) -> str**, not **(req: StoryRequest, tone: str) -> str**. Adding a **tone** parameter “since Module 4 will need it” pre-decides the wrong design — Module 4 will correctly place **tone-and-safety** in the system prompt (a different SDK parameter, a different file). PRD §1.5 rule 3 (YAGNI / Rule of Three). When in doubt, leave the signature as narrow as the current module's caller needs.

No tests in V1. The function is testable, but V1 doesn't ship with a test framework. During a live demo, the maintainer runs the function in a REPL — that's the demonstration of testability. PRD §1.5 prohibitions list (no tests in this curriculum).

Defend It

We didn't change behaviour — the same inputs produce the same kind of prompt, which produces (statistically) the same kind of story. What did we gain by extracting *compose_story_prompt* into its own function in its own file?

Don't ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Looking back — Modules 2 and 3 together

These two modules are best understood as a pair. Each has its own single fundamental, but the arc between them is the bigger lesson.

Module 2 — domain follows the schema, not the UI. When the kind of work changes (from “log a Q&A pair” to “generate a story for a specific child”), the schema (**StoryRequest** with four named fields) and the endpoint name (**/story**, not aliased from **/ask**) change first. The UI is downstream of the schema, not the other way around. You also see three layers of validation — browser required, Pydantic 422, handler 400 on `.strip()` — each catching a class of input the others can't. The deliberately ugly inline f-string in the **/story** handler is a debt Module 3 pays.

Module 3 — prompt composition is itself a piece of code with rules and care. The prompt is not a string buried in a handler — it's a designed object that deserves a name (**compose_story_prompt**), a home (**app/prompt.py**, peer to **app/schemas.py**, not under **services/**), and a shape that's testable in isolation. Behaviour is unchanged; what's gained is a visible, testable, REPL-pokeable design surface for prompt iteration. A pure refactor, defended by `git diff` reading + REPL inspection rather than green tests.

Side-by-side:

	Module 2	Module 3
Where prompts live	Inline f-string, ~3 lines, in the /story handler	app/prompt.py , a function callable from a REPL

	Module 2	Module 3
Why it matters	The schema names the new domain; the prompt should too — but doesn't yet	Now the prompt has a file-level home — Module 4 will add a sibling for safety
What stays the same	The /story API contract, the form, the SDK call	Everything user-visible — same payload, same kind of story
The temptation to resist	Adding <code>compose_story_prompt()</code> already (jumps the lesson)	Adding a tone: <code>str</code> parameter “since Module 4 will need it” (wrong design surface)

The bigger lesson chained across both modules: naming follows the domain at every layer — schema, endpoint, file location, function signature. Each module changes one layer; the next module's lesson depends on the previous layer being right.

The cumulative payoff lands in Module 4. By then you have `StoryRequest` (Module 2's schema), `compose_story_prompt` in `app/prompt.py` (Module 3's user-prompt surface), and a missing sibling — `app/system_prompt.py` — which Module 4 introduces. Two files, two prompts, two design surfaces, each evolving on its own clock.

If you generated a Module 2 or Module 3 story and noticed it was short (~80 words instead of bedtime-appropriate length): that's not a bug. The system prompt in `app/services/gemini_service.py` is still V1's leftover Q&A framing — “Answer in one short paragraph (under 80 words)”. The model is honouring it. Module 4 is when that constant moves out and gets rewritten for child-safe bedtime tone, and your stories grow into proper length and shape.

Module 4 — Strengthen the system prompt for child safety + bedtime tone

The notional machine

An LLM call has two prompt parameters that look similar but evolve at very different rates. The user prompt (`contents=` in the SDK) varies per request — it carries the form fields the parent just typed. The system prompt (`config.system_instruction=`) is constant across requests — it tells the model who it is, what it does, and what constraints it always honours. The model receives both. The model's output is conditioned on both. The system prompt is sent to Google's servers exactly once per request, but the content of the system prompt is decided once, in code, and stays the same until the developer changes it.

In V1 (and through Modules 1–3 of this course), the system prompt was a Q&A leftover — “You are a concise, helpful assistant. Answer in one short paragraph...”. The model politely ignored it for bedtime-story requests because the user prompt's “write a bedtime story” dominated. Module 4 fixes that debt with five explicit constraints (length cap, no violence, gentle resolution, sensory detail, address by name) plus a refuse-with-grace clause that softens edge-case requests rather than stonewalling them.

The analogy

A new hire with a job description tacked to their desk. The job description (system prompt) is the standing instructions: who they are, what they do, what they always do, what they refuse to do. The user message (user prompt) is the question they just got from a colleague. The hire reads the job description as context every time they answer; the colleague's question varies, the desk's note doesn't. A weak job description (“be helpful”) gets a generic hire who will write whatever you ask. A specific job description (“you write bedtime stories for young children, max five paragraphs, gentle endings”) gets a hire who applies those constraints to every question, including the questions that try to push past them.

Where the analogy breaks: a real hire pushes back on conflicting instructions; an LLM blends them statistically. If the user prompt directly contradicts the system prompt, the model is somewhere between honour the system

prompt, honour the user, and split the difference. That's why "gently steer toward a safe alternative without lecturing the user" is in the constraints — it tells the model how to resolve conflicts.

What goes, what comes

Goes: - The whole `SYSTEM_PROMPT = "You are a concise..."` constant in `app/services/gemini_service.py`. The line that built `generate_config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)` stays byte-identical — but now it imports `SYSTEM_PROMPT` from a different file.

Stays: `app/prompt.py`, `app/main.py`, `app/schemas.py`, `app/templates/index.html`, `requirements.txt`, `.env.example`. The user prompt, the SDK call wiring, the validation paths, the form — none of them know that the system prompt's content shifted. That's the point of the system-vs-user separation: each evolves without touching the other.

Comes: - A new file `app/system_prompt.py` (sibling to `app/prompt.py`) — the system prompt as a distinct design surface. - An import in `app/services/gemini_service.py`: `from app.system_prompt import SYSTEM_PROMPT`. - Five behavioural constraints in the system prompt's text, plus a refuse-with-grace closing clause.

The code

`app/system_prompt.py` (new file):

```
SYSTEM_PROMPT = (
    "You are writing short bedtime stories for young children to be read aloud "
    "by a parent at the end of the day.\n"
    "\n"
    "Always follow these rules:\n"
    "- Keep the story short --- five paragraphs at most.\n"
    "- Use calm, gentle language. No violence, no scary themes, no characters "
    "in distress at the end.\n"
    "- The story must end with all named characters safe, comforted, and at rest.\n"
    "- Use concrete sensory details (sounds, textures, light) so the child can "
    "picture each scene.\n"
    "- Refer to the child by the name given in the prompt; do not address the "
    "reader as 'you'.\n"
    "\n"
    "If the prompt asks for content that violates these rules, gently steer the "
    "story toward a safe alternative without lecturing the user."
)
```

`app/services/gemini_service.py` — top of file before (Modules 1–3):

```
import os
from google import genai
from google.genai import types
from google.genai import errors as genai_errors
from fastapi import HTTPException

GEMINI_API_KEY = os.environ["GEMINI_API_KEY"]
GEMINI_MODEL = "gemini-2.5-flash-lite"
SYSTEM_PROMPT = (
    "You are a concise, helpful assistant. "
    "Answer in one short paragraph (under 80 words). "
    "If you don't know, say so plainly."
)

client = genai.Client(api_key=GEMINI_API_KEY)
```

```
generate_config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)
```

After (Module 4):

```
import os
from google import genai
from google.genai import types
from google.genai import errors as genai_errors
from fastapi import HTTPException

from app.system_prompt import SYSTEM_PROMPT

GEMINI_API_KEY = os.environ["GEMINI_API_KEY"]
GEMINI_MODEL = "gemini-2.5-flash-lite"

client = genai.Client(api_key=GEMINI_API_KEY)
generate_config = types.GenerateContentConfig(system_instruction=SYSTEM_PROMPT)
```

The `generate_config = ...` line is byte-identical. The wiring to the SDK didn't change. What changed is who owns the prompt content. `gemini_service.py` is now a thin adapter that knows the SDK's contract; `app/system_prompt.py` is the design surface for the model's behaviour.

Trace it in execution order

When a `/story` request arrives:

1. At program start (already happened): `app/services/gemini_service.py` is imported, which triggers `from app.system_prompt import SYSTEM_PROMPT` — Python reads `app/system_prompt.py`, finds the constant, returns to `gemini_service.py`. The `generate_config` object is built once with that constant baked in.
2. Request arrives: handler runs `compose_story_prompt(payload)` — this is the user prompt only (the form fields). The system prompt is not in this string.
3. **`call_gemini(prompt)`** runs: the SDK call sends both prompts to Google in the same HTTP request — the user prompt as `contents=...` and the system prompt as `config.system_instruction=...` (which was set when `generate_config` was built).
4. Google's model processes the conversation: it reads the system prompt as standing instructions, then reads the user prompt as the current request, then generates an answer conditioned on both.
5. Response returns: the story body is in `response.text`. The story should now be visibly shorter, gentler, more paragraphed, and address the child by name.

The interesting timing: the system prompt is decided once (at module import) and reused on every request. Changing it requires editing `app/system_prompt.py` and restarting the server. The user prompt is built per-request from form fields. Two timescales, two design surfaces, two files.

Predict before you run

- Run the same form payload through Module 3 and Module 4 with the same Gemini key. Will the stories be identical? Will they be the same kind of different they'd be if you ran Module 3 twice?
- POST `/story` with a deliberately scary plot — `"plot": "the monster chases them through the woods at night"`. Predict: will the model refuse outright ("I cannot write that"), produce a graphic horror story, or steer toward a softer version?
- If you delete the "do not address the reader as 'you'" line from the system prompt and re-run, what changes about the story style?
- If you move the closing "gently steer..." clause to the top of the system prompt instead of the bottom, will the model honour it more or less reliably? (This one is genuinely an experiment — try it.)

Run + verify

```
cd ../module_04_safety_system_prompt
source ../../venv/bin/activate
cp .env.example .env # if you don't already have one in this folder
uvicorn app.main:app --reload

# Plumbing --- same /story shape:
curl -s -X POST http://localhost:8000/story \
  -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"an owl and a fox",
      "setting":"enchanted forest","plot":"looking for the moon"}'
# Expected: {"story":"<a SHORTER, GENTLER, MORE-PARAGRAPHED story than Module 3
↳ produced>"}

# Constraint-honouring soft check --- try a deliberately scary plot:
curl -s -X POST http://localhost:8000/story \
  -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"a monster",
      "setting":"a dark forest","plot":"the monster chases them through the woods
↳ at night"}'
# Expected: a story that includes the monster and the chase but ends with everyone
# safe and at rest. NOT "I cannot write that." NOT graphic violence.

# All-in-one:
./scripts/verify_module_4.sh
```

The verify script can't auto-check non-deterministic content. Manual inspection rubric: the benign-payload story should be ≤ 5 paragraphs, gentler, and address the child by name. The scary-payload story should soften — include the monster, end with comfort, no refusal. If the model refuses outright with “I cannot help with that”, the “gently steer” clause is being missed; sharpen its placement.

Why this design

File location: sibling, not nested, not bundled. The user prompt and the system prompt go to different SDK parameters (contents= vs config.system_instruction=) and they evolve at different rates (per-request vs constant). Putting both in app/prompt.py would conflate them; putting SYSTEM_PROMPT under services/ would mis-classify it as SDK configuration. app/system_prompt.py peer to app/prompt.py mirrors the SDK's two-parameter design. PRD §1.5 rule 4 (build for maintainability — file structure follows the API).

Plain Python multi-line string, not a dataclass / **dynaconf** template / separate **.txt** file. The constraints are five bulleted lines a parent could read and a developer could change in one diff. PRD §1.5 rule 1 (simplicity over demonstration — Python f-strings are clearer than Jinja2 for a five-bullet template).

Refuse-with-grace, not refuse-with-warning. Production wisdom. A parent's child says plot=the dragon eats everyone. A naive system prompt produces “I'm sorry, I can't help with that” — the parent and child are confused. A graceful system prompt produces a story where the dragon is hungry but ends up sharing a feast — request honoured, safety honoured, no lecture. The closing clause is the difference between “an LLM with a content filter” and “a thoughtful child-safe writer.”

The system prompt is in code, not in environment. Promoting SYSTEM_PROMPT to an env var would uncouple it from the code that depends on it. The prompt evolves with the model, the wording, the safety rules — those are code-shaped decisions, not environment-shaped. Pydantic-settings-style env-driven prompts are on PRD §1.5's prohibition list for the same reason.

Defend It

Why does the “no violence, no scary themes” constraint go in the system prompt instead of being added as a sentence at the end of *compose_story_prompt*’s user prompt?

Don’t ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 5 — Update the UI for parents reading aloud

The notional machine

A page is a layout on top of a typography on top of a state machine. The layout decides where things sit (form left, story right). The typography decides how they read (serif at 1.2 rem with a 38ch line cap so a tired parent’s eyes track the next line). The state machine decides what’s visible (story-empty / story-loading / story-rendered, plus a body-level story-active class so cross-pane styling can be CSS-driven).

CSS Grid is the layout engine. Six lines suffice:

```
.layout {  
  display: grid;  
  grid-template-columns: 1fr 1.6fr;  
  gap: 2rem;  
}  
@media (max-width: 700px) {  
  .layout { grid-template-columns: 1fr; }  
}
```

That’s the whole layout. No framework, no preprocessor, no design-token JSON.

The analogy

A children’s picture book vs. a finance Bloomberg terminal. Both are pages. One is meant to be read aloud at 8 PM by a tired parent who hasn’t slept properly in weeks; the other is meant to be scanned for tickers by a trader at noon. Same medium, totally different design. Module 5 redesigns the page to match what a parent reading aloud is actually doing: long single column of serif text, capped at line lengths the eyes don’t lose, on a soft cream pane that doesn’t glare at midnight.

Where the analogy breaks: a picture book has illustrations. The story-pane doesn’t (V1 doesn’t generate images). The typography is doing the work the illustrations would have done.

What goes, what comes

Goes: - The flat <body>-with-form-then-divs structure in `index.html`. - All the V1 history-list styling in `style.css` — `#history li`, `#history .meta`, etc. Dead since Module 2 (the V1 history panel was deleted there); its styles were dead weight.

Stays: `app/main.py`, `app/services/`, `app/prompt.py`, `app/system_prompt.py`, `app/schemas.py`, `requirements.txt`, `sql/`. The backend bytes do not move — this is a frontend-only module.

Comes: - A <header> + <main class="layout"> containing two <section> panes — form-pane and story-pane. - Three explicit story states in CSS — story-empty, story-loading, story-rendered — and a body-level story-active class for cross-pane styling. - A rewritten `style.css` built around Grid layout, serif typography for the story body, `max-width: 38ch` for read-aloud line cadence, `body.story-active .form-pane { opacity: 0.6 }` for visual focus, and a `@media (max-width: 700px)` collapse for phones.

The code

`app/templates/index.html` — substantive shape:

```

<main class="layout">
  <section class="form-pane" aria-label="Story details">
    <p class="hint">Tell us about the story you'd like to read tonight.</p>
    <form id="story-form"> ... 4 inputs + button ... </form>
    <p id="error" class="error" role="alert"></p>
  </section>
  <section class="story-pane" aria-label="Story" aria-live="polite">
    <div id="story-output" class="story-empty">
      <p class="placeholder">Your story will appear here.</p>
    </div>
  </section>
</main>

```

Two `<section>` elements with semantic class names give CSS something to grid against. `aria-live="polite"` on the story pane lets screen readers announce the story when it arrives, without interrupting the parent.

app/static/style.css — five typography/layout decisions:

```

.layout {
  display: grid;
  grid-template-columns: 1fr 1.6fr;    /* form narrower, story wider */
  gap: 2rem;
}

#story-output {
  font-family: Georgia, "Iowan Old Style", serif;
  font-size: 1.2rem;
  line-height: 1.75;
  max-width: 38ch;
  margin: 0 auto;
}

body.story-active .form-pane { opacity: 0.6; }
body.story-active .form-pane:hover,
body.story-active .form-pane:focus-within { opacity: 1; }

@media (max-width: 700px) {
  .layout { grid-template-columns: 1fr; }
}

```

Each rule is tied to a use case:

- `1fr 1.6fr` — form narrower, story wider. The story is the read object; it gets the larger pane.
- `Georgia, "Iowan Old Style", serif` — signals “this is a story to be read.” Form inputs stay system-ui sans-serif.
- `max-width: 38ch` — comfortable read-aloud line cadence. At 80ch the eyes overshoot; at 25ch the rhythm is choppy.
- `line-height: 1.75` — a tired parent’s eyes don’t lose the next line.
- `body.story-active .form-pane { opacity: 0.6; }` — the form recedes when a story is active. Hover or focus restores it (the parent can still generate another story without a click).

Trace it in execution order

When a parent clicks Generate story with a story already on the right pane:

1. The submit handler runs in the browser. It sets `body.classList.add("story-active")` and the `#story-output` div’s class to `story-loading`.

2. CSS reacts immediately: `body.story-active .form-pane` matches → `opacity: 0.6` is applied to the form pane. `.story-loading` swaps in the spinner.
3. The fetch fires. While it's pending, the page is in the loading state.
4. Response arrives. The handler sets `#story-output` to `story-rendered` and writes paragraphs.
5. CSS reacts again: the `story-rendered` rules apply (the cream pane background, the serif typography on `#story-output`). The serif text appears in the right pane.
6. If the parent moves their cursor over the form pane, `:hover` matches and `body.story-active .form-pane:hover { opacity: 1 }` overrides the dim. They can edit a field and submit again without an explicit “edit” mode toggle.

The state machine is implicit in CSS classes. No JavaScript framework manages it; the browser's existing class-cascading behaviour does the work.

Predict before you run

- Resize the browser to 600px wide. What does the `.layout` rule do? (Hint: the media query cuts in at 700px.)
- The form pane fades to 0.6 opacity when a story is active. What happens if you remove the `:hover` override? Try it (just in DevTools).
- The story-pane's `max-width: 38ch` caps line length even when the pane is wider. Predict: if you delete the `max-width` rule, does the story now fill the pane? Does that make it more or less readable?
- The CSS uses `body.story-active` to drive cross-pane styling. The class is added by JavaScript when a story is generated. What happens if you remove the JS line that adds it? Trace through what no longer works.

Run + verify

```
cd ../module_05_story_ui
source ../../venv/bin/activate
cp .env.example .env # if you don't already have one in this folder
uvicorn app.main:app --reload

# Backend regression --- /story unchanged from Module 4:
curl -s -X POST http://localhost:8000/story -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"an owl and a fox",
      "setting":"enchanted forest","plot":"looking for the moon"}'

# Frontend renders new structure:
curl -s http://localhost:8000/ | grep -E 'class="(layout|form-pane|story-pane)"'

# CSS ships grid + serif:
curl -s http://localhost:8000/static/style.css | grep -E
↪ '(grid-template-columns|Georgia|@media)'

# All-in-one:
./scripts/verify_module_5.sh
```

Open `http://localhost:8000`. Manual inspection rubric: form sits left, story sits right at ≥ 700 px? After generating a story, does the form pane visibly dim? Story rendered in serif on a cream pane? Resize narrower than 700px — does the layout stack? If any answer is no, the corresponding CSS rule needs work.

Why this design

No CSS framework. Six lines of Grid CSS handle a two-column responsive layout. PRD §1.5 rule 1 (simplicity over demonstration) — Bootstrap or Tailwind would buy nothing the lesson requires.

No third grid column for Module 6's "Past stories" panel. Module 6 owns its layout decisions; reserving an empty column now would be the YAGNI violation. PRD §1.5 rule 3 (Rule of Three).

Form-pane dim driven by CSS, not JS inline styles. The rule belongs in CSS so it's discoverable and overridable. Hover and focus behaviour come for free; doing this in JS would mean re-implementing the browser's hover machinery. PRD §1.5 rule 4 (build for maintainability).

max-width: 38ch on the story body. Read-aloud cadence is a real ergonomic concern: a parent's eyes track better with shorter lines. The unit `ch` (character width) means the cap follows the typography — change the font size, the cap follows automatically. `vw`-based or pixel-based caps would not.

Frontend-only module. The backend doesn't know the UI changed. That's the whole point — separating presentation from logic means the same `/story` endpoint serves the V1 single-column app, this two-column app, and any future redesign. No backend refactor, no schema migration, no API change.

Defend It

Why does the form pane fade to opacity 0.6 when a story is active, instead of disappearing entirely or staying at full opacity?

Don't ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 6 — Story library (re-hear yesterday's story)

The notional machine

Persistence is a database server holding rows on its own disk, addressed by your app over a connection. When your `uvicorn` process restarts, the rows survive because they live in Postgres's data files, not your app's memory. Module 6 reintroduces that machinery — last seen in V1's Q&A app — but with a new table whose columns name the bedtime-story domain: `child_name`, `characters`, `setting`, `plot`, `body`, `model_name`, `created_at`. A composite index on (`child_name`, `created_at` DESC) answers the one query that matters: all stories for this child, latest first.

Click-to-rehear is the user-value: clicking a saved story renders the verbatim body from the database. Zero Gemini calls. The child gets the same words they fell asleep to last night — important because LLMs are non-deterministic and regenerating from the same form fields would produce a different story.

The analogy

A filing cabinet next to the parent's desk. Each new story they write goes into the cabinet, filed under the child's name with the date. Yesterday's story isn't in their head — they made it once, the words are gone — but it's in the cabinet. Pulling out the right folder and reading the same words again is fundamentally different from writing a new one.

Where the analogy breaks: a real filing cabinet doesn't have a composite index. Postgres's index is the difference between flip through every folder until you find the child's name (sequential scan) and open straight to the right tab (index seek). The schema is making product decisions visible — "we will always retrieve by `child_name`, sorted by date descending" — and Postgres is rewarding the honest schema.

What goes, what comes

Goes: - The V1 `interactions` table — dead in the codebase since Module 2, kept in the database through Modules 3–5 so rollbacks worked, dropped here because the new migration is the right place. PRD §1.5 rule 2 (take away, not add).

Stays: `app/prompt.py`, `app/system_prompt.py`, `app/services/gemini_service.py`, `requirements.txt`, `.env.example`. The user prompt, the system prompt, and the SDK call are all unchanged. The deploy-config files (Module 7) don't exist yet.

Comes: - sql/002_create_stories.sql — the migration: DROP TABLE IF EXISTS interactions; CREATE TABLE stories (...); CREATE INDEX ... - app/services/story_service.py (sibling to gemini_service.py) — save_story(req, body) and fetch_recent_stories(child_name, limit=5). The shape mirrors the V1 interaction_service.py Module 2 deleted; the names follow the new domain. - A StoredStory Pydantic class in app/schemas.py — for the typed retrieval response. StoryRequest and StoryResponse are unchanged. - One new line in /story: save_story(payload, story_text) after call_gemini returns. - A new GET /stories?child_name=... endpoint. - A “Past stories” <aside> panel inside the form pane, populated on child_name blur, with click-to-rehear that serves the body from the DOM (not from a re-fetch).

The code

sql/002_create_stories.sql:

```

DROP TABLE IF EXISTS interactions;
CREATE TABLE stories (
    id SERIAL PRIMARY KEY,
    child_name TEXT NOT NULL,
    characters TEXT NOT NULL,
    setting TEXT NOT NULL,
    plot TEXT NOT NULL,
    body TEXT NOT NULL,
    model_name TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_stories_child_name_created_at ON stories (child_name, created_at
↪ DESC);

```

app/services/story_service.py:

```

def save_story(req: StoryRequest, body: str) -> int:
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO stories (child_name, characters, setting, plot, body,
↪ model_name) "
                    "VALUES (%s, %s, %s, %s, %s, %s) RETURNING id",
                    (req.child_name.strip(), req.characters.strip(),
↪ req.setting.strip(),
                    req.plot.strip(), body, GEMINI_MODEL),
                )
                story_id = cur.fetchone()[0]
                conn.commit()
                return story_id
    except psycopg.Error:
        raise HTTPException(status_code=502, detail="Postgres is not reachable. Check
↪ your database connection.")

def fetch_recent_stories(child_name: str, limit: int = 5) -> list[StoredStory]:
    with get_conn() as conn:
        with conn.cursor(row_factory=dict_row) as cur:
            cur.execute(
                "SELECT id, child_name, characters, setting, plot, body, model_name, "
                "to_char(created_at, 'YYYY-MM-DD HH24:MI:SS') AS created_at "
                "FROM stories WHERE child_name = %s ORDER BY created_at DESC LIMIT %s",

```

```

        (child_name.strip(), limit),
    )
    return [StoredStory(**row) for row in cur.fetchall()]

```

The shape mirrors the V1 `interaction_service.py` Module 2 deleted — same `try/except psycpg.Error`, same with `get_conn()` context-manager pattern, same `dict_row` factory, same narrow exception catch translating to `HTTPException(502)`. The persistence pattern is the lesson; the names follow the new domain.

app/main.py:

```

@app.post("/story", response_model=StoryResponse)
def story(payload: StoryRequest):
    if not payload.child_name.strip() or not payload.plot.strip():
        raise HTTPException(status_code=400, detail="Please fill in at least the
            ↪ child's name and the plot.")
    story_text = call_gemini(compose_story_prompt(payload))
    save_story(payload, story_text)
    return StoryResponse(story=story_text)

@app.get("/stories", response_model=list[StoredStory])
def stories(child_name: str):
    if not child_name.strip():
        raise HTTPException(status_code=400, detail="child_name query parameter is
            ↪ required.")
    return fetch_recent_stories(child_name)

```

Frontend additions in app/templates/index.html:

```

<aside id="recent-stories" class="recent-stories" hidden>
    <h2>Past stories</h2>
    <ul id="recent-list"></ul>
</aside>

```

```

async function loadRecentStories() {
    const name = document.querySelector('[name="child_name"]').value.trim();
    if (!name) { aside.hidden = true; return; }
    const items = await (await
        ↪ fetch(`/stories?child_name=${encodeURIComponent(name)}`).json());
    if (items.length === 0) { aside.hidden = true; return; }
    list.innerHTML = items.map(it => `<li><button class="recent-item"
        data-body="${escapeHtml(it.body)}">
        <span class="recent-plot">${escapeHtml(it.plot)}</span>
        <span class="recent-date">${escapeHtml(it.created_at)}</span>
        </button></li>`).join("");
    list.querySelectorAll(".recent-item").forEach(btn =>
        btn.addEventListener("click", () => renderStory(btn.dataset.body)));
    aside.hidden = false;
}
document.querySelector('[name="child_name"]').addEventListener("blur",
    ↪ loadRecentStories);

```

The `data-body` attribute on each button carries the saved story body. The click handler reads it and passes to `renderStory`. No second fetch, no Gemini call. The body is already in the response from `/stories?child_name=...`; we hold onto it in the DOM.

Trace it in execution order

When the parent generates a new story (POST /story):

1. Validation runs (Module 2's three layers).
2. `compose_story_prompt(payload)` builds the user prompt (Module 3).
3. `call_gemini(prompt)` calls Gemini with the user prompt + the system prompt (Modules 1 + 4). Returns `story_text`.
4. NEW: `save_story(payload, story_text)` opens a Postgres connection, INSERTs the row (RETURNING id), commits, closes. If Postgres is down, raises `HTTPException(502)` and the response is never sent. One handler, one transaction-shaped outcome: either we have a story to return and a row in the DB, or we have neither.
5. The handler returns `StoryResponse(story=story_text)`. The frontend doesn't get the new `story_id` because no client needs it.

When the parent's cursor leaves the `child_name` field (blur event):

1. `loadRecentStories()` runs. Reads the value, trims it.
2. If blank, hides the panel and returns.
3. Otherwise, fetches `/stories?child_name=...`. The handler runs `fetch_recent_stories(child_name, limit=5)` — Postgres uses the composite index to seek straight to the child's rows in date-descending order, returns up to 5.
4. The frontend renders the list as `<button>` elements with `data-body` carrying the story body verbatim (escaped).
5. Each button gets a click handler that calls `renderStory(btn.dataset.body)` — no fetch — the story renders instantly in the story-pane.

When the parent generates another story for the same child:

1. The POST flow runs again, persisting a new row.
2. The frontend's submit handler also calls `loadRecentStories()` after the response arrives, so the panel updates without a page refresh.

Predict before you run

- Generate a story for `child_name="Aisha"`. Then generate another for `child_name="Bola"`. Then run `loadRecentStories` for Aisha (by blurring the field with Aisha in it). What does the panel show? What's the SQL query that ran?
- Click an older story in the panel. Open DevTools Network tab. How many requests fire? (The lesson is in the answer: zero requests to `/story` or `/stories`.)
- Stop Postgres mid-test. POST `/story` again. What error code, what response body? (Hint: trace through `save_story` — which exception is caught, which is raised?)
- A row was saved with `child_name=" Aisha "` (whitespace). Does a GET `/stories?child_name=Aisha` find it? Why or why not? (Look at the `.strip()` calls in `save_story` and the WHERE clause in `fetch_recent_stories`.)

Run + verify

```
cd ../module_06_story_library
source ../../venv/bin/activate
cp .env.example .env          # if you don't already have one in this folder

# Reminder: `DATABASE_URL` must be exported in your *shell* for the psql lines below
# to work.
# If you opened a fresh terminal since Module 0 (the macOS "default interactive shell
# is now zsh"
# banner is the giveaway), the export from that earlier session is gone --- re-export
# now, or set
```

```

# it persistently per the recommended option in Module 0's setup. Symptom if missing:
# `psql: ... FATAL: database "<your-username>" does not exist`.
# export
↪ DATABASE_URL=postgresql://postgres:postgres@localhost:5432/llm_question_log

psql "$DATABASE_URL" -f sql/002_create_stories.sql      # one-time migration: DROP
↪ interactions, CREATE stories
uvicorn app.main:app --reload

# Generate (now persists):
curl -s -X POST http://localhost:8000/story -H "Content-Type: application/json" \
  -d '{"child_name":"Aisha","characters":"an owl","setting":"forest","plot":"moon"}'

# Confirm row landed (same `DATABASE_URL` shell-export caveat as above):
psql "$DATABASE_URL" -c "SELECT id, child_name, plot, LEFT(body, 60) FROM stories
↪ ORDER BY id DESC LIMIT 1"

# Retrieval:
curl -s "http://localhost:8000/stories?child_name=Aisha"

# All-in-one:
./scripts/verify_module_6.sh

```

Open the page, generate two stories for the same `child_name`, click an older one in the panel. With DevTools Network tab open: zero requests to `/story`, the story-pane fills with the saved body. If the click triggers a `/story` call, the click handler is regenerating instead of serving from `data-body` — fix the JS.

Why this design

Store the **body**, not just the inputs. This is the heart of the module's Defend-It. Three reasons in one breath: regenerating from (`child_name`, `characters`, `setting`, `plot`) gives a different story (LLMs are non-deterministic); regenerating costs a Gemini call (latency, quota, money); the child wants the same words they fell asleep to last night. Storing both the inputs and the body costs disk space (essentially free) and buys all three.

Composite index (**`child_name`, `created_at` DESC**). The exact query the new endpoint runs is all rows for one child, latest first. A single `child_name` index would force in-memory sort. A single `created_at` index would force scan-and-filter. The composite index serves the query as an index seek with no extra work. The schema is making the access pattern visible.

save_story runs after **call_gemini** returns. If Gemini fails, **call_gemini** raises `HTTPException(502)` and the persist line is never reached. Half-rows on Gemini failure would be worse than no row. PRD §1.5 rule 1 (simplicity over demonstration — one transaction shape).

Click-to-rehear serves from **data-body**, not a second fetch. The body was already in the response from `/stories?child_name=...`; refetching by ID would be wasted round-trips. The DOM holds the body; the click renders it. PRD §1.5 rule 2 (take away, not add).

Drop the V1 **interactions** table here, in Module 6's migration. Modules 3–5 needed it to survive in the DB so rollbacks would work. Module 6 owns the schema deletion because its migration is the natural place — same migration that creates the new table. PRD §1.5 rule 2 — dead schema is dead code, deletion is the doctrine.

Defend It

Why does the *stories* table store both the four input fields (*child_name*, *characters*, *setting*, *plot*) AND the generated *body* — couldn't we just store the inputs and re-generate when needed?

Don't ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Module 7 — Deploy: Vercel frontend + Render backend

The notional machine

A production deployment splits the app into two processes living in two different places. Vercel serves the frontend — three static files (`index.html`, `style.css`, `script.js`) — from its global CDN. There's no Vercel-side Python; the browser downloads the files, the JS runs in the browser, and the browser makes cross-origin `fetch()` calls to the backend. Render runs the backend — long-running `uvicorn` process bound to whatever `$PORT` Render assigns — and provisions a managed Postgres database alongside it. `DATABASE_URL` and `GEMINI_API_KEY` are environment variables in Render's runtime, not in source control.

The backend stops serving HTML. The `app/templates/` and `app/static/` directories are deleted; `jinja2` is dropped from `requirements.txt`; the `/` route is gone. CORS middleware lets the browser (running on `your-app.vercel.app`) talk to the backend (running on `bedtime-story-api.onrender.com`) — without it, the browser refuses the cross-origin request.

The frontend's `BACKEND_URL` is a hardcoded JS constant — one line you edit per deploy. No build step, no env-var injection, no runtime configuration trickery. Vercel detects the static structure (because of `vercel.json`) and ships the files; Render detects the Python service (because of `Procfile` + `render.yaml`) and runs `uvicorn`.

The analogy

A restaurant with a delivery service. The kitchen (Render) cooks the food and stays open between orders. The delivery cars (Vercel) carry the menus to the customer's house. The customer (browser) reads the menu, calls the kitchen on the phone (CORS-allowed cross-origin request), the kitchen cooks, the delivery brings it back. Two different operations in two different places, talking over a defined protocol.

Where the analogy breaks: a real kitchen doesn't spin down after 15 minutes of no orders; Render's free-tier web service does. The first request after a spin-down takes 30+ seconds — a cold start, not a bug. Render's free Postgres expires 30 days after creation (with a 14-day grace period before deletion); free web services across your workspace are capped at 750 instance-hours per calendar month. Both are deliberate cost trade-offs; both are real properties you'll explain to a parent who tries the app on a Sunday morning after no one's used it for an hour, or to yourself when you discover your stories table is suddenly unreachable a month after the cohort demo.

What goes, what comes

Goes: - `app/templates/` — entire directory deleted. The HTML is now in `frontend/`, served by Vercel. - `app/static/` — entire directory deleted. CSS is now in `frontend/`. - `jinja2` from `requirements.txt` — the backend no longer renders HTML. - The `/` GET handler in `app/main.py` — and the `Jinja2Templates`, `HTMLResponse`, `StaticFiles` imports. - The “uvicorn serves everything” local-dev workflow — replaced by two-terminal local dev.

Stays: - `/story`, `/stories`, `/healthz` handlers — byte-identical to Module 6. - `app/services/`, `app/schemas.py`, `app/prompt.py`, `app/system_prompt.py`, `sql/`. All unchanged. - The stories table from Module 6 — carries to Render's managed Postgres via the same `sql/002_create_stories.sql` script.

Comes: - A new top-level `frontend/` directory containing `index.html`, `style.css`, `script.js`. The HTML and CSS are lifted verbatim from Module 6's `app/templates/index.html` and `app/static/style.css`; the JS is extracted from the inline `<script>` and prefixed with the `BACKEND_URL` constant. - CORS middleware on the backend: `app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)`. - Three deploy-config files: `Procfile` (Render web service start command), `render.yaml` (Render Blueprint — service + database + env vars), `vercel.json` (Vercel — `outputDirectory: "frontend"`, no build step).

The code

`app/main.py` — backend changes:

```

# Before (Module 6):
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from fastapi.staticfiles import StaticFiles
# ... mount /static, define templates, define index handler ...

# After (Module 7):
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
# ... no static mount, no templates, no index handler ...

app = FastAPI(title="Bedtime Story Generator")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # V1 simplicity. Tighten to your Vercel domain in
    ↪ production,
    allow_methods=["*"],
    allow_headers=["*"],
)
# /story, /stories, /healthz handlers UNCHANGED.

```

frontend/script.js — top of file:

```

// Replace this with your Render service URL after deploying the backend.
// For local development: keep "http://localhost:8000" and run uvicorn locally.
const BACKEND_URL = "http://localhost:8000";

// Every fetch() call uses ${BACKEND_URL}:
//   fetch(`${BACKEND_URL}/story`, ...)
//   fetch(`${BACKEND_URL}/stories?child_name=...`)

```

Procfile:

```
web: uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

render.yaml:

```

services:
- type: web
  name: bedtime-story-api
  runtime: python
  plan: free
  buildCommand: pip install -r requirements.txt
  startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
  envVars:
  - key: DATABASE_URL
    fromDatabase:
      name: bedtime-story-db
      property: connectionString
  - key: GEMINI_API_KEY
    sync: false # set in the Render dashboard; never committed

databases:
- name: bedtime-story-db
  plan: free

```

vercel.json:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "buildCommand": null,
  "outputDirectory": "frontend",
  "cleanUrls": true
}
```

Trace it in execution order

A production request from a parent in their browser at `https://bedtime-story.vercel.app`:

1. Browser hits Vercel: the parent navigates to the URL. Vercel's CDN serves `frontend/index.html` from the nearest edge.
2. HTML loads: the browser parses, fetches `style.css` and `script.js` (both from Vercel's CDN). The `<form>` renders.
3. Parent submits the form: the JS handler runs, builds the JSON body, calls `fetch(`${BACKEND_URL}/story`, ...)` where `BACKEND_URL` is `https://bedtime-story-api.onrender.com`.
4. Browser pre-flights (because cross-origin POST with `Content-Type: application/json`): sends `OPTIONS /story` to the Render URL. The CORS middleware on the backend responds with `Access-Control-Allow-Origin: *`, `Access-Control-Allow-Methods: *`, `Access-Control-Allow-Headers: *`.
5. Browser sends the real POST: the request lands on the Render service. If the service was idle, Render is now spinning up the container — the parent waits 30+ seconds. If warm, the request is hot.
6. Backend handles the request: same code path as Module 6 — Pydantic validation → handler check → compose → Gemini → save → response.
7. Render's Postgres: `save_story` opens a connection to the managed database (URL from `DATABASE_URL` env var, set automatically by `render.yaml`'s `fromDatabase`), INSERTs the row.
8. Response goes back: through Render's load balancer, across the internet, into the browser. The JS handler renders the story.
9. Subsequent requests: the service is warm; latency is back to normal. Until the next 15-minute idle period.

The local-dev workflow uses two terminals because there are two services. Terminal 1 runs `uvicorn app.main:app --reload` (the backend on `:8000`). Terminal 2 runs `python -m http.server 5173 --directory frontend` (the frontend on `:5173`). The browser hits `http://localhost:5173`; the JS calls `http://localhost:8000`; CORS lets the cross-origin request through. The `BACKEND_URL` constant points at `localhost` in dev, gets edited to the Render URL before commit when you're ready to ship.

Predict before you run

- You commit the deploy configs and push. Vercel auto-deploys; Render auto-deploys. The Vercel URL loads the frontend, but submitting the form fails with a CORS error in DevTools. What did you forget?
- After both services are deployed, the first `/story` POST takes 35 seconds. The second takes 2 seconds. What's happening?
- You set `GEMINI_API_KEY` in `vercel.json` instead of in Render's env. Where does the key actually go, and what error do you see?
- You leave `BACKEND_URL = "http://localhost:8000"` in the deployed `script.js`. What error does the deployed frontend produce, and where is it visible?
- Render's free Postgres expires 30 days after creation (then a 14-day grace period before deletion). The deploy still works on day 31 in degraded form — just not the parts that touch Postgres. What error code does `/story` return? What error code does `/stories` return? After day 45 the database is gone entirely; what changes?

Run + verify

Local dev (two terminals, both inside `dist/module_07_deploy_vercel/`):

```
# Terminal 1 --- backend
cd dist/module_07_deploy_vercel # or `cd ../module_07_deploy_vercel` from Module 6
source ../../venv/bin/activate
cp .env.example .env # if you don't already have one in this folder
uvicorn app.main:app --reload

# Terminal 2 --- frontend (in a fresh terminal, also inside this dist folder)
cd dist/module_07_deploy_vercel
python -m http.server 5173 --directory frontend

# Browser: http://localhost:5173
```

Production deploy — high-level summary below; the click-by-click step-by-step lives in docs/deploy_guide.md. Read this summary for the why, follow the deploy guide for the how.

1. Render: Sign in, New → Blueprint, connect the GitHub repo. Render reads render.yaml and provisions both the web service and the managed Postgres. After the first deploy, set GEMINI_API_KEY in the service's Environment tab. Open the database's external shell and run `psql "$DATABASE_URL" -f sql/002_create_stories.sql` to apply the schema. (Render's external shell sets `$DATABASE_URL` for you — no shell export needed there. The shell-export discipline only applies to your local terminal, where it's still required for any `psql` command you run against your local Postgres.)
2. Vercel: Sign in, Add New → Project, import the repo. Vercel reads vercel.json and deploys frontend/. No build step.
3. Wire: in frontend/script.js, change `const BACKEND_URL = "http://localhost:8000"` to `const BACKEND_URL = "https://bedtime-story-api.onrender.com"` (your actual Render URL). Commit. Vercel auto-redeploys.
4. Verify: open the Vercel URL. Generate a story. Confirm it persists (`SELECT id, plot FROM stories ORDER BY id DESC LIMIT 1` from Render's shell). Click a saved story in the panel — confirm zero /story requests in DevTools.

For step-by-step screens, button labels, common gotchas, and the exact text to paste into Render's database shell, see docs/deploy_guide.md. That guide is the click-by-click companion to this section — same destination, more hand-holding.

```
# Local pre-deploy verify:
./scripts/verify_module_7.sh
```

The script checks: frontend/ exists with three files; app/templates/ and app/static/ deleted; CORS middleware wired; jinja2/HTMLResponse/StaticFiles imports gone; Procfile, render.yaml, vercel.json present; BACKEND_URL constant in script.js; /healthz returns OK; CORS preflight to /story returns the expected headers; / and /static are 404; /story, /stories, /healthz work.

Why this design

Vercel for static, Render for long-running — not Vercel-everything serverless. PRD §3 locks this. The V1 mental model is “uvicorn keeps a Python program alive between requests” — the receptionist who never goes home. Re-learning that to deploy would conflate “what’s a server” with “how to deploy a server.” Vercel-everything serverless would force exactly that re-learning: cold starts on every request, connection-pool churn, function-execution time limits. The split-stack option preserves the V1 mental model and uses two free tiers that each do one job well.

BACKEND_URL as a hardcoded JS constant, not a build-step env var. Three options were on the table: a Vercel build step that templates the URL from a Vercel env var (rejected — adds a build step for one constant, bundles “frontend tooling” into Module 7); the hardcoded constant (chosen — one file, one variable, students can grep for it); a same-origin reverse proxy via Vercel (rejected — defeats the split-stack and adds the proxy as infrastructure). The constant ships with `localhost:8000` so local dev just works. PRD §1.5 rule 1 (simplicity over demonstration).

CORS `allow_origins=["*"]` for V1. Production tightening to a specific Vercel domain is a named follow-up, not a Module 7 requirement. V1 stays simple to make demo failures debuggable; the bedtime-story API has no auth, no PII, no tokens that cross-origin malice could exfiltrate. Tightening `allow_origins` introduces a typo-risk that breaks every request without the parent ever seeing a useful error message. PRD §1.5 rule 1.

Three small files for three deploy concerns. `Procfile` says how Render starts the backend. `render.yaml` says what Render provisions and how secrets/connections wire up. `vercel.json` says what Vercel deploys and how. A future maintainer asking “how does this run in production?” has three small files to read. Combining them into a single `deploy.yaml` would buy nothing — the targets are different, the formats are different. PRD §1.5 rule 4 (build for maintainability).

Delete `app/templates/` and `app/static/`, don’t keep them “for local dev.” Local dev now uses two terminals; the index route would be misleading dead code. Two ways to serve the same page is a recipe for confused students and out-of-sync drift. PRD §1.5 rule 2 (take away, not add).

This is the final V1 module. The app is on the internet. By Module 7’s end, you’ve shipped a small but real production service: a frontend on a CDN, a backend on a long-running host, a managed database, and a working LLM call from a parent’s living-room browser at 8 PM.

Defend It

Why do we deploy the frontend as static files on Vercel and the backend as a long-running uvicorn process on Render — instead of running both on Vercel as serverless functions?

Don’t ask your AI partner to answer this. Reasoning through it yourself is the assessment.

Where to next

You’ve shipped V1. The bedtime-story app generates child-safe stories from a four-field form, persists them per-child for re-hearing, and runs on the internet. Five honest directions from here:

1. Tighten production. Replace `allow_origins=["*"]` with your specific Vercel domain. Add Render observability (logs, metrics). Move `BACKEND_URL` to a Vercel build-step env var so dev/prod don’t share the same `script.js`. None of these are V1 concerns — they’re hardening, not new fundamentals.
2. Add streaming. Gemini supports streaming responses; the SDK has `client.models.generate_content_stream(...)`. Streaming is its own fundamental: chunked responses, incremental rendering, mid-stream error handling. Deferred from V1 by PRD §7 because it would have hidden the basics. A worthwhile next module.
3. Add per-family accounts. V1 treats `child_name` as free-text — two families with a child named Aisha see each other’s stories. Authentication is its own fundamental — a real one, not a tutorial-fake one. Pick OAuth (sign-in-with-Google) over rolling your own; pick a session cookie over a JWT.
4. Feed past stories back into the prompt. Currently the system prompt is constant. A V2 idea: when generating a new story for a child, include the last 1–2 stories’ first paragraphs as context in the system prompt — the model can match tone. This earns the “context window management” fundamental.
5. Build the next thing. You can call a hosted LLM through an SDK, write a constrained system prompt, persist generated artifacts, and deploy a split frontend/backend stack. That’s the foundation for any number of next apps — a recipe assistant, a meeting-notes summariser, a personalised study tool. Pick one with a real user (yourself, your kid, your team) and ship its V1.

Whichever you pick: keep the doctrine. One fundamental per module, take away rather than add, name what’s there, don’t abstract until duplication earns it. The doctrine isn’t unique to this course; it’s how to keep small codebases readable as they grow into larger ones.

About this course

The structure of this crash course is research-backed. The eight-section per-module pattern is drawn from converging findings in learning science:

- Notional machines (du Boulay, 1986; Sorva, 2013): students who can describe the runtime mental model of code outperform students who can describe the syntax. The “notional machine” section is built so the runtime model arrives before the code.
- Self-explanation effect (Chi et al., 1989; Chi, 2018): readers who pause to predict and explain before running code build durable understanding. The “predict before you run” section is the operationalisation.
- PRIMM (Sentance, Waite, Kallia, 2019; expansions through 2025): Predict, Run, Investigate, Modify, Make. The crash course’s predict + trace + run + verify steps map directly to the first four.
- Faded scaffolding for adult learners (Springer 2025; ACM TOCE 2025): explicit prompt fading produces durable independence. The dist READMEs across the curriculum exhibit fading (~6–8 prompts in early modules, ~2 prompts in late modules); the crash course is the self-paced echo of that structure.
- AI partner as scaffold, not crutch (IJSE 2025, “Tool, tutor, or crutch?”): students who are coached not to ask the AI for answers to assessment-grade questions retain better than students who use the AI freely. The Defend-It questions and the “Don’t ask your AI partner” warnings operationalise this.

You don’t need to read any of these papers to use the course. The structure is doing the work whether you notice it or not.

If you’re an instructor adapting this course, the per-session loop in *INSTRUCTOR_GUIDE.md* (Pre-Demo Setup → Per-Module Demo Sections → V1 complete → Three-Surface Teaching Model → Per-Session Loop → Cohort Delivery Process) is the live-cohort companion to the eight-section pattern in this crash course. The conversion-course note at the top of *INSTRUCTOR_GUIDE.md* is required reading before demoing any module past 0.